

MLflow, Docker, CI/CD & Testing

Detailed Q&A Revision Guide

Nityaa Kalra | August 2026

SECTION 1: MLFLOW EXPERIMENT TRACKING

Q1: What is MLflow and what problem does it solve?

MLflow is an open-source platform for managing the ML lifecycle. Without it, experiments are chaotic: you run 50 experiments, change hyperparameters, and have no idea which run produced which model or which config achieved the best F1.

The four main components:

1. **MLflow Tracking**: log parameters, metrics, and artifacts per run
2. **MLflow Projects**: package code in reproducible format
3. **MLflow Models**: standard format for packaging models
4. **MLflow Registry**: versioned model store with lifecycle stages

Q2: How do you use MLflow Tracking in a training script?

```
import mlflow
import mlflow.pytorch

mlflow.set_experiment('stance-detection-slm')
with mlflow.start_run(run_name='qwen3-1.7b-lora-r16'): # Log hyperparameters
    mlflow.log_params({'model': 'Qwen3-1.7B', 'lora_r': 16, 'lr': 2e-5, 'epochs': 3,
                      'dataset': 'SemEval-2016'})
    for epoch in range(3): # ... training ...
        mlflow.log_metrics({'train_loss': loss, 'val_macro_f1': f1, 'val_accuracy': acc },
                           step=epoch)
    # Log model
    mlflow.pytorch.log_model(model, 'stance_model')
    # Log confusion matrix as artifact
    mlflow.log_artifact('confusion_matrix.png')
```

Q3: What is the MLflow Model Registry and what are the lifecycle stages?

The Model Registry is a central model store with versioning and lifecycle management. You register a model from a run, then promote it through stages:

Stage	Meaning
None	Just registered, not ready
Staging	Being tested, candidate for production

Production	Live, actively serving requests
Archived	Retired, kept for auditing

```
from mlflow.tracking import MlflowClient client = MlflowClient() # Register model
from run result = mlflow.register_model( f'runs:{run_id}/stance_model',
'StanceClassifier' ) # Promote to staging client.transition_model_version_stage(
name='StanceClassifier', version=result.version, stage='Staging' ) # Load production
model model = mlflow.pytorch.load_model( 'models:/StanceClassifier/Production' )
```

Q4: What is the difference between **log_param** and **log_metric** in MLflow?

log_param: logs a single key-value configuration (hyperparameter). Logged once per run. E.g., learning rate, batch size, model name.

log_metric: logs a numeric value that can change over time. Takes an optional step argument to track evolution over epochs. E.g., loss, F1 score.

You can then compare runs in the MLflow UI and plot metric curves.

SECTION 2: DOCKER

Q5: What is Docker and why does ML need it?

Docker packages your code, dependencies, and environment into a container — a lightweight, isolated unit that runs the same everywhere. 'It works on my machine' is the classic problem Docker solves.

For ML specifically: your model depends on exact versions of PyTorch, CUDA, tokenizers, etc. Docker freezes this environment so your inference service runs identically in development, staging, and production.

Q6: Write a Dockerfile for a Python ML inference service.

```
# Use official Python base image FROM python:3.11-slim # Set working directory inside container WORKDIR /app # Copy and install dependencies first (for layer caching) COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt # Copy application code COPY . . # Expose the port the app runs on EXPOSE 8000 # Command to run CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why copy requirements.txt before code? Docker caches each layer. If requirements don't change, it won't reinstall packages even if you change your Python code.

Q7: What are the most important Docker commands?

```
docker build -t stance-api:latest . # build image from Dockerfile docker run -p 8000:8000 stance-api # run container, map port docker run -d stance-api # run in background (detached) docker ps # list running containers docker logs # view container output docker exec -it /bin/bash # shell into container docker stop # stop container docker rm # remove container docker images # list local images docker push myrepo/stance-api:v1.0 # push to registry
```

Q8: What is the difference between an image and a container?

Image: read-only blueprint (like a class in OOP). Built from Dockerfile. Stored in a registry (Docker Hub, AWS ECR).

Container: running instance of an image (like an object). Can be started, stopped, deleted. Multiple containers can run from the same image.

SECTION 3: CI/CD WITH GITHUB ACTIONS

Q9: What is CI/CD and how does it apply to ML?

CI (Continuous Integration): automatically test and validate code on every push. Catches bugs early, enforces code quality, ensures the main branch is always stable.

CD (Continuous Delivery/Deployment): automatically deploy tested code to staging or production. In ML, this might mean building a new Docker image, deploying to an API server, or registering a new model version.

Q10: Write a GitHub Actions CI workflow for an ML project.

```
# .github/workflows/ci.yml name: ML CI Pipeline on: push: branches: [main, develop] pull_request: branches: [main] jobs: lint-and-test: runs-on: ubuntu-latest steps: -
```

```
name: Checkout code uses: actions/checkout@v3 - name: Set up Python uses:  
actions/setup-python@v4 with: python-version: '3.11' - name: Install dependencies  
run: pip install -r requirements.txt - name: Format check (black) run: black --check  
. - name: Lint (flake8) run: flake8 src/ - name: Type check (mypy) run: mypy src/ -  
name: Run tests run: pytest tests/ -v --cov=src --cov-report=xml - name: Upload  
coverage uses: codecov/codecov-action@v3
```

SECTION 4: PYTEST & TESTING FOR ML

Q11: How do you write unit tests for ML code with pytest?

ML testing is different from regular software: you test data pipelines, model outputs, preprocessing functions, and metric computations — not just pure logic.

```
# tests/test_metrics.py import pytest import torch from src.metrics import
compute_macro_f1, compute_class_shift def test_macro_f1_perfect_predictions(): preds
= [0, 1, 2, 0, 1] labels = [0, 1, 2, 0, 1] assert compute_macro_f1(preds, labels) ==
1.0 def test_macro_f1_all_wrong(): preds = [1, 2, 0, 1, 2] labels = [0, 1, 2, 0, 1]
f1 = compute_macro_f1(preds, labels) assert f1 == 0.0 def test_class_shift_sign(): #
Model predicts more 'none' than gold preds = [2, 2, 2, 0] labels = [0, 1, 2, 0] shift
= compute_class_shift(preds, labels, class_idx=2) assert shift > 0 # over-predicts
none def test_model_output_shape(): model = StanceClassifier(768, 256, 3) dummy_input
= torch.randn(8, 128, 768) # batch, seq, hidden output = model(dummy_input) assert
output.shape == (8, 3) # batch x num_classes
```

Q12: What are pytest fixtures and why are they useful?

Fixtures are reusable setup functions — they create objects (models, datasets, tokenizers) that multiple tests need, without repeating setup code.

```
import pytest from transformers import AutoTokenizer @pytest.fixture def tokenizer():
return AutoTokenizer.from_pretrained('bert-base-uncased') @pytest.fixture def
sample_batch(): return { 'texts': ['I favor climate action', 'I oppose this policy'],
'labels': [0, 1] # favor, against } # Test uses both fixtures def
test_tokenizer_output(tokenizer, sample_batch): encoding =
tokenizer(sample_batch['texts'], padding=True) assert 'input_ids' in encoding assert
len(encoding['input_ids']) == 2
```

Q13: What is test coverage and what's a reasonable target?

Test coverage measures the percentage of your code lines executed by tests. 100% is not the goal — untestable code and edge cases make this impractical.

Reasonable targets: 70-80% for ML projects. Prioritize coverage on: data preprocessing, metric computation, model I/O shapes, config loading.

```
pytest tests/ --cov=src --cov-report=term-missing # Output shows which lines are NOT
covered: # src/metrics.py 95% Missing lines: 47, 82
```

Q14: How do you handle slow tests (e.g., tests that load models)?

Mark slow tests with a custom marker so you can skip them in fast CI runs:

```
# conftest.py def pytest_addoption(parser): parser.addoption('--run-slow',
action='store_true') @pytest.fixture def run_slow(request): return
request.config.getoption('--run-slow') # In test file @pytest.mark.slow def
test_full_model_inference(): model = load_full_model() # slow — loads 1.7B params ...
# Run fast tests only (CI default) pytest tests/ -m 'not slow' # Run all including
slow (nightly) pytest tests/ --run-slow
```